05 Ayush

Comparison, critical analysis and conclusion

Fuzzy-Logic-Based QoS Control in oneM2M (Zyane & Et-Tousy, FiCloud 2026)

Your job

About **5 minutes**. Put John's numbers in context: what were they compared with, do they add up, what did our own re-simulation show, does it scale, and what are the weak spots. Then close the talk and run Q&A. You took over the harder results material from Part 4: the static-vs-fuzzy figure (Fig. 4), Table IV, the cost breakdown and AWS pricing (Table II).

Your slides

Slides 27–34. When the slide colour turns pink, you're on.

Opening line

"Thanks John. Those are strong numbers. Let's see what they are compared with, whether they add up, and where the paper is weaker."

Closing line

"To sum up: a small, readable fuzzy controller inside oneM2M that improves speed, reliability and cost under overload. Convincing as a prototype, but the evidence is still one edge node and one cloud VM. Thank you; we're happy to take questions."

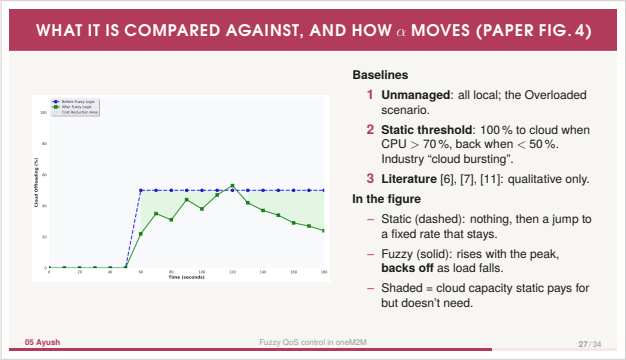
Arithmetic you can show on demand

Cost: $(2.1667 - 1.2242)/2.1667 = 0.435$ RTT: $(8.50 - 3.75)/8.50 = 0.559$
Success: $(92.5 - 77.5)/77.5 = 0.194$ relative $=$ +15 percentage points absolute
Medium: $0.5417 \to 0.3575$ (-34 %) Peak: $0.5417 \to 0.3033$ (-44 %) Heavy: $0.5417 \to 0.5583$ ($+3$ %)
t2.medium: \$0.0464/h $\Rightarrow$ \$0.1667 $\approx$ 3.6 instance-hours; \$0.0942 $\approx$ 2.0 h (our arithmetic)

Q&A routing

Controller maths $\to$ Ashutosh. oneM2M / architecture $\to$ Manoj. Motivation / related work $\to$ Raj. Results numbers $\to$ John can repeat them, you handle any "but" question.

Slide 27 ~70 s



Left: paper Fig. 4, offloading ratio over time, static (dashed blue) vs. fuzzy (solid green), shaded saving. Right: the three baselines and how to read the figure.

Say: "Thanks John. Those are strong numbers. Let's see what they are compared with, whether they add up, and where the paper is weaker."

Say: "On the right: the paper compares against three things. First, unmanaged: everything stays local. That is John's overloaded scenario, a default Mobius install, the worst case."

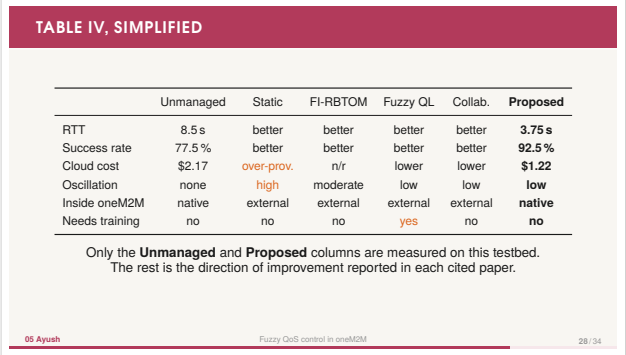
Say: "Second, a static threshold: send 100 % of traffic to the cloud when CPU goes over 70 %, and come back when it falls below 50. That is the typical industry 'cloud bursting' rule."

Say: "Third, published methods: FI-RBTOM, a fuzzy rule-based offloading model; a fuzzy Q-learning scheme; and a load-balance-aware collaborative offloading strategy. Important: those are compared only qualitatively, from what their papers report."

Say: "The figure on the left shows how much traffic each method sends to the cloud over time. The static rule, dashed, does nothing until the CPU limit, then jumps to a fixed level and stays there, even after the load drops."

Say: "The fuzzy controller, solid, rises with the peak and then backs off as load falls. The shaded area is cloud capacity the static rule pays for but does not need. That is where the cost saving comes from."

Slide 28 ~40 s



	Unmanaged	Static	FI-RBTOM	Fuzzy QL	Collab.	**Proposed**
RTT	8.5 s	better	better	better	better	**3.75 s**
Success rate	77.5 %	better	better	better	better	**92.5 %**
Cloud cost	$2.17	over-prov.	n/r	lower	lower	**$1.22**
Oscillation	none	high	moderate	low	low	**low**
Inside oneM2M	native	external	external	external	external	**native**
Needs training	no	no	no	yes	no	**no**

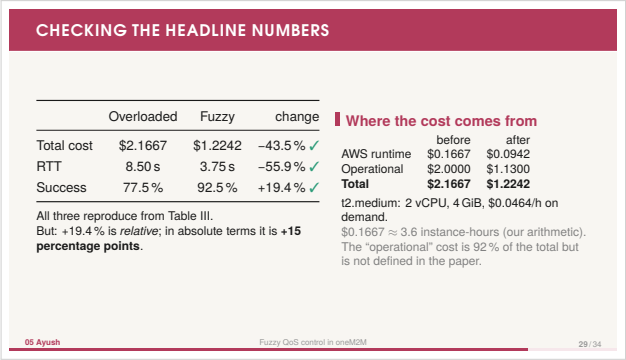
Table IV simplified: six methods, six criteria.

Say: "This is the paper's Table IV, simplified. Read the last column: 3.75 seconds RTT, 92.5 % success, $1.22, low oscillation, native in oneM2M, no training needed."

Say: "Static thresholds improve on unmanaged, but over-provision and oscillate: the bang-bang problem. The fuzzy Q-learning approach does well but needs training. And all the others run outside oneM2M."

Say: "But be careful: only the unmanaged and proposed columns are measured. The others are just the direction of improvement their own papers reported, on different setups. So this table positions the work; it does not prove it is better."

Slide 29 ~45 s



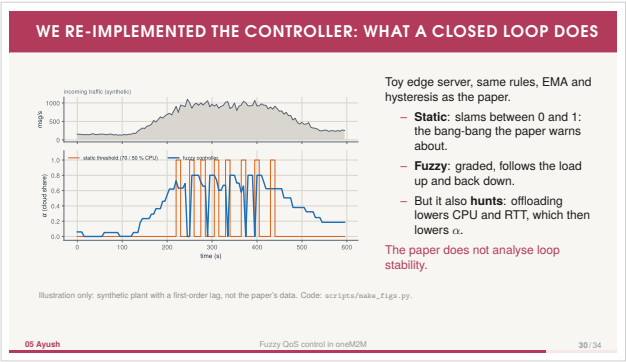
Headline numbers recomputed from Table III; cost breakdown; AWS t2.medium price.

Say: "We checked the headline numbers against Table III, and all three reproduce: 43.5 % cost, 55.9 % RTT, 19.4 % success."

Say: "One nuance: the 19.4 % is relative. In absolute terms, success goes from 77.5 to 92.5 %, that's 15 percentage points. Still good, but say it precisely."

Say: "On cost: the instance is a t2.medium at about 4.6 cents an hour. The AWS runtime part of the bill matches a few instance-hours. But most of the bill, 92 %, is an 'operational' cost the paper never defines. So the 43.5 % is mostly a saving on a cost model we cannot inspect."

Slide 30 ~50 s



Our re-simulation: synthetic load on top; α for static (orange steps) vs. fuzzy (blue).

Say: "We re-implemented the controller from the paper's equations and Table I, and put it in a closed loop with a toy edge server. This is our illustration, not the paper's data."

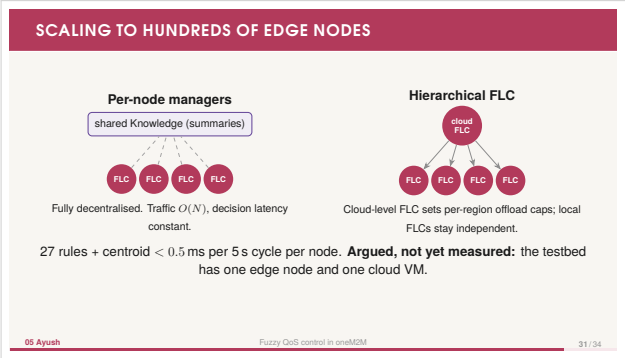
Say: "The static rule, orange, slams between 0 and 1, exactly the bang-bang the paper describes."

Say: "The fuzzy controller, blue, is graded and follows the load up and back down, which matches the paper's story. But we also see it hunting. When it offloads, CPU and RTT fall, so on the next cycle it sees a calmer server and offloads less, and then load comes back. The smoothing and hysteresis help, but the paper never analyses this loop's stability. That is a fair question for the authors."

Q: *Isn't that just your toy model?*

A: Yes, the exact shape depends on our plant. The point is structural: two of the controller's three inputs (CPU, RTT) are measured after offloading, so its own action changes its inputs. The paper does not discuss this.

Slide 31 ~40 s

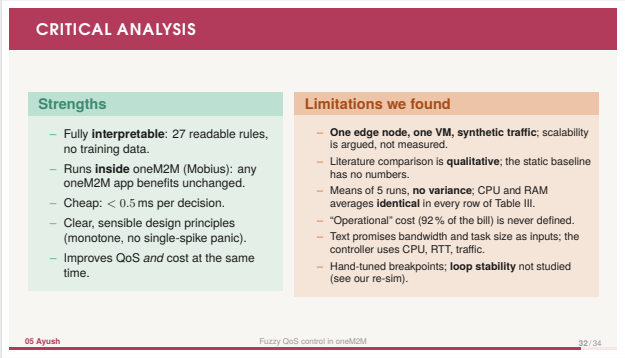


Two scaling patterns: per-node FLCs with shared knowledge; a hierarchical cloud FLC setting caps.

Say: "What about hundreds of edge nodes? The paper proposes two patterns. Each node runs its own controller and only shares summaries: communication grows linearly with nodes, decision time stays constant. Or a hierarchy: a cloud-level fuzzy controller sets per-region offloading caps, while each node still decides locally."

Say: "The controller is cheap, under half a millisecond every 5 seconds, so it won't be the bottleneck. But note: this is argued, not measured. The testbed has one edge node and one cloud VM."

Slide 32 ~60 s



Strengths (green) vs. limitations we found (orange).

Say: "Overall assessment. Strengths: it's fully interpretable, 27 rules you can read, no training data. It runs inside oneM2M, so any oneM2M app benefits unchanged. It's cheap. The design principles are sensible. And it improves QoS and cost together."

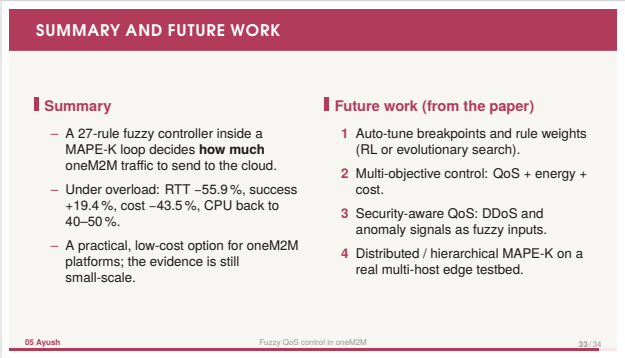
Say: "Limitations. One edge node, one VM, synthetic traffic. Literature comparison is qualitative, and the static baseline has no numbers. Results are means of 5 runs with no variance. Curiously, CPU and RAM averages are identical in every scenario of Table III. The operational cost is undefined. The text promises bandwidth and task size as inputs, but the controller uses CPU, RTT and traffic. And the breakpoints are hand-tuned with no stability analysis."

Tip: If time is short, say the first two limitations and the stability point only.

Q: *Anything editorial?*

A: Some references are duplicated (e.g. [1] and [22], [2] and [9], [4] and [16], [6] and [19]) and one citation renders as "[?]" in the scalability section.

Slide 33 ~40 s



Summary on the left, four future-work items on the right.

Say: "To sum up: a 27-rule fuzzy controller inside a MAPE-K loop decides how much oneM2M traffic to send to the cloud. Under overload it cuts RTT by 56 %, raises success by 19 % relative, cuts cost by 43 %, and brings CPU back to 40–50 %."

Say: "The authors plan four next steps: auto-tune the rules with reinforcement learning or evolutionary search; optimise QoS, energy and cost together; add security signals like DDoS detection as fuzzy inputs; and run a distributed, hierarchical version on a real multi-host edge testbed, which would answer most of our concerns."

Slide 34 –



Thank-you slide with the five of us and the full citation.

Say: "A small, readable fuzzy controller inside oneM2M that improves speed, reliability and cost under overload. Convincing as a prototype, but the evidence is still one edge node and one cloud VM. Thank you; we're happy to take questions."

Harder questions you might get

Q: *Would a well-tuned PID controller do the same?*

A: Possibly for a single input. The fuzzy approach handles three inputs with readable rules and no model; the paper does not compare against PID, which is a fair gap.

Q: *Why does heavy load cost more with the controller?*

A: The paper calls it a deliberate trade-off: it offloads more to absorb bursts (heavy load has $\times 1.5$ bursts) and keep QoS, offset by savings at medium and peak.

Q: *Is 3.75 s RTT actually good for IoT?*

A: Not in absolute terms; seconds-level RTT means heavy queueing even in the normal case (3.25 s). The claim is relative improvement on this testbed.

Q: *So is ML just worse?*

A: Not worse, different trade-off. ML needs data that matches the real deployment and retraining when it changes;

fuzzy rules are readable and tweakable on the spot. Ashutosh's slide 13 covers it; the authors' future work uses RL to tune the rules, not replace them.

Q: *How does it compare to fuzzy Q-learning numerically?*

A: It doesn't, numerically: Table IV is qualitative for the literature methods.